Final Project: Pick a Project

Computer Science - Week 10

Oct 3, 2026 - Version 0.0.2

Please make sure to your UD **email** AND **name** is below. Unless your UD emails are included in this table, then you will not earn any points for this assignment when it is graded! **Every person should submit their own individual worksheet**.

Talk about each of the questions with a friend or our LLM (or both), but **make sure the words you enter below are yours** (with the obvious exceptions notwithstanding). Brainstorming is best done in full sentences, and the main goal here is brainstorming. It's worth reviewing the information in this series of pages in the Drafter documentation to gain a better understanding of the process.

Role	Name	Email
Developer (You)	Ryan Williamson	raw@udel.edu
Friend (optional)	Emma Highberger	ejhigh@udel.edu

Part 1: The Goal & Prior Implementations

1. **Identify the Goal:** What specific goal are you trying to address? Think about your passions, hobbies, and academic interests. The latter are an excellent candidate to focus on. Some majors have suggestions listed below: these were provided by faculty in those departments. Other majors could consider a focus tied to a big final assignment in one of their recent courses in their major.
    - **Biomedical Engineering** majors may consider one of the following:
        - If you have started (or completed) the Cell/Tissue Design final project in BMEG100, design an organoid, which involves combining a cell type, biomaterial, and testing method. Have the website include a description and diagram of your design concept, a testing method, a needs statement, and an impact statement relevant to their organoid. Also include a link to a paper that supports your idea and a paragraph describing how that paper supports your design concept.

        - Otherwise, create a website related to your BMEG100 Lab Report 1, *Cytoskeletal and Nuclear Staining Lab Report Guidelines*, linked here (University of Delaware account required to access).
    - **Chemistry** and **Chemical Engineering** majors may consider creating an automated chemistry assistant that takes balanced reaction equations, starting reactant masses, and a harvested product amount to determine the limiting reactant, theoretical yield, and percent yield based on principles from Chemistry 2e (OpenStax) — Chapter 4: Chemical Reactions and Stoichiometry. To avoid manually compiling atomic mass tables or writing custom string tokenizers, use the ChemPy library to parse molecular formulas into exact molecular weights and verify reaction stoichiometry against compound data from the NIH PubChem Database. The program outputs an organized breakdown of limiting reagent status, product yields, and remaining masses of unreacted species. Also use the heat of the reaction to determine if it is exothermic or endothermic, and compute the heat flow per mole of the product.
    - Majors in the **Electric & Computer Engineering** department may consider a steganography project that hides messages in images, and then retrieves those messages from uploaded images.
    - **Mechanical Engineering** majors may consider a project that somehow uses both the NumPy and SciPy Python libraries.

I think it would be so awesome and so cool to create something like a Wiki webpage. When I play video games, I find myself really wanting to learn the niche ins and outs of the game and learn how it works, but I find that in most cases those resources are not available. Thus, I've always wanted to create my own resource for compiling, connecting and documenting things, like a well-organized wiki

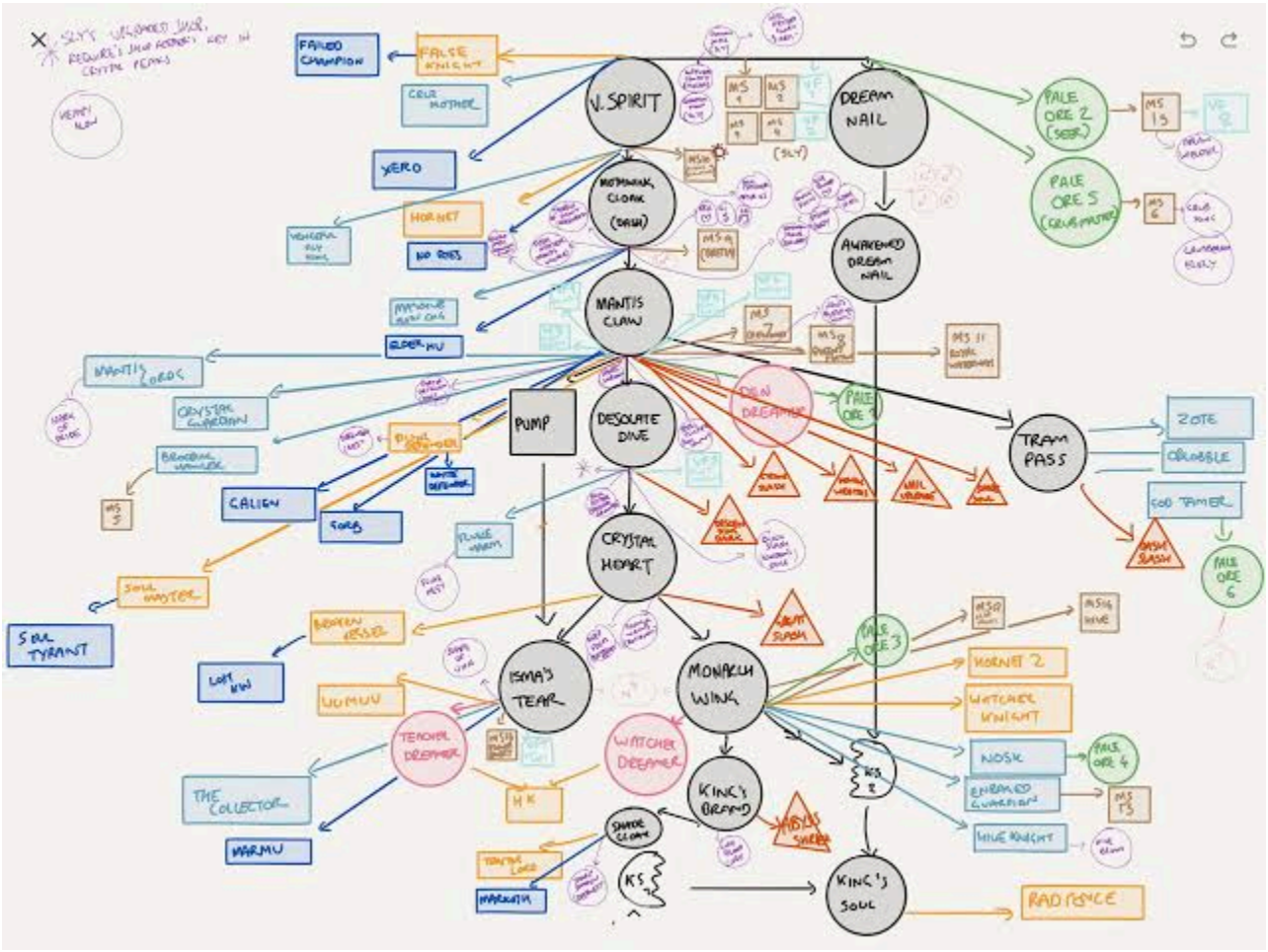
2. **Prior Implementations:** Search the internet. What existing tools or websites currently attempt to address it?

There are Wikis for some very popular games, of which I'd certainly take inspiration from. Particularly, I enjoy the layout and depth of the Hollow Knight Wiki and Stardew Valley Wiki.

3. **Your Implementation:** Briefly describe the legitimate purpose or functionality of your site. How would you want this web application to be used?

Well, while I'm certainly *pretty* deep into a few fandoms, I know there are sooo many people equally or more invested than I. So, I'd like to make a website to be able to easily look up

things relating to a game and understand how it is actually built. I'm thinking of things like a search feature, links on specific keywords that take you to explanation pages, and charts/ diagrams. Ts (this stuff) really gets my mind working, and I know I'm only scratching the surface compared to the wealth of information out there.



Part 2: The Input-Process-Output (IPO) Model

Think about the fundamental flow of your program.

1. **The Core Goal:** Every program has three main phases: taking input, processing data, and presenting output. In one sentence, what is the ultimate goal your program will accomplish for the user? Be specific and use CS1 concepts.

I want to compile information on a video game through verifiable player feedback and playtesting, organize it, and create an extensive wiki site surrounding it.

2. Input: What specific information will the user need to provide to your program? (e.g., a city name, a numerical budget, a list of hobbies).

Mostly just searching/ interacting with the website/ clicking links. Maybe consider contributing information.

3. Processing: At a high level, how will the computer manipulate or evaluate that data? (e.g., calculate the average, filter out options that are too expensive, compare the input against a database).

It will connect the user with relevant information.

4. Output: What will the user get back? (e.g., a recommendation, a calculated safety factor, a formatted schedule).

Details involving game mechanics, backend coding, gameplay systems– anything the psycho user could possibly want to dig into and pick apart their favorite game.

Part 3: Data Abstraction

Identify the "nouns" of your problem.

1. **Representing Reality:** Abstraction is the process of representing the real world inside a computer by ignoring unnecessary details. What are the core "things" (nouns) your program needs to track? (e.g., *If building a recipe app, the nouns are "Recipe" and "Ingredient"*).

Most likely less concerned about music/ art/ graphics of a game (other than raw documentation), and more with progression/ technical possibilities/ hitboxes/ code.

2. **Identifying Properties:** For the most important "thing" you listed above, what 3 to 4 specific details do you actually need to solve your problem, and what details will you abstract away?
    - *Details I need:* (e.g., *Ingredient name, weight, cost*)
    - *Details I will ignore:* (e.g., *Ingredient color, brand, expiration date*)

Present the systems and mechanics of the game in a far more detailed way than is actually let on in-game. Things such as specific variable names from digging around in the code can

be ignored– the thing that matters is the output, and what specifically it makes possible.

Part 4: Challenges, AI Strategy, & Goals

1. **Difficulties:** What parts of this project seem the most difficult or technically complex?

Probably the compilation of data, which will need to be thorough and take time.

2. **Coding Strategy:** If you will be using an LLM to help code, how will you ensure you understand the generated code and avoid "hallucinations" or "shallow learning" (copying without understanding)? If you won't be using an LLM to help code, this project might take about 4 hours a week to complete, so list each of the days and times in which you work on the project.

I can test this by comparing it to my knowledge of whatever game I choose to be the subject of this deep-dive, and by opposing descriptions of systems against one another in the documentation of the wiki to ensure there aren't any contradictions. I can test things in the game of my own accord.

3. **Goals:** What is your ultimate goal for this project? (e.g., *Impress the graders with a creative UI, build something I can use in my other classes, learn Drafter deeply*).

Honestly, a lot of it is to alleviate my personal burden of having to scrap for information and having limited certainty about its credibility. However, I also think it would be good for myself to understand the lengthy progress that goes into it, and understand how I can pick apart systems and present them understandably.

Part 5: Research & Brainstorming Conversation

Have a conversation with a Generative AI tool (like ChatGPT or Claude) OR a friend about your project idea. Do not ask for code. Instead, use the AI/friend as a sounding board to research your problem.

1. **The Initial Prompt/Question:** Ask the AI or your friend to help you brainstorm. (e.g., *"I want to build a simple web tool to help students track their study hours.*

What are 3 common problems students face when tracking their time, and what are 3 simple features a basic tool should have?"). Paste your prompt below:

What frustrates you when trying to find information online?

2. **The Response:** Briefly summarize the best ideas the AI or your friend suggested. What existing solutions or prior apps did they mention?

Getting information that seems complete but is really lacking sufficient context, or not finding something that matches the issue I'm facing.

3. **The Pushback:** You must reply to the AI/friend to narrow the scope when it's over-ambitious. Pick the most complex feature they suggested and explain why it is too complicated for an introductory programming project. Ask them for a simpler alternative. *Paste your response here:*

I believe that creating a full scale library on the level of a wiki page for a game is rather ambitious for an introductory coding project. This scope can be mitigated by either centering about a very small subject (a small game), or by limiting what I am specifically trying to map out (instead of trying to explain all systems and mechanics, maybe focus on a specific question: as in, for a nonlinear game, "What would all possible manners of progression look like?/ What are all of the different ways to beat the game?"-- in effect, anything that could narrow the scope a bit.

4. **Reflection:** Based on this conversation, how has your project idea changed or become more focused?

I think the discussion was helpful for refining my scope. I'm still brainstorming what I want to specifically set my sights on, but I think I have a good judgement on my capabilities now.

Part 6: Resource Planning

1. What resources will you use if you get stuck?

Honestly, probably just asking my friends, or professor/ TAs. I generally don't have issues with standing up for myself/ asking for help, so I think I will be okay. Discord is there too, and LLMs.

2. If your LLM provides code that causes a Drafter or Bakery error, or if you run into a problem during deployment, what is your step-by-step debugging plan to resolve it?

Probably examine the code and the error message (if provided) for a while, reread it a few times, take a small break and come back to it, and then ask somebody to look at it with a fresh set of eyes. Or, if it's an LLM issue, it might help to simply refer the details of the error to the LLM and see if it gives you a course of action to make progress in resolving it.

Part 7: LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

I didn’t use an LLM for this, just spoke to a couple of my buddies from class :)

Part 8: Submit

In Google docs, with this document open, please click on *File* > *Download* > *Microsoft Word (.docx)*, and make sure you put that download somewhere you can find it again. Then, upload it to the Canvas assignment about selecting or brainstorming your final project.

When you're done with this assignment, review this page of the Drafter documentation to better prepare for the next step.